

```

int Index KMP(SString S, SString T, int next[]) {
    int i=1, j=1;
    while(i<=S.length&& j<=T.length) {
        if(j==0 || S.ch[i]==T.ch[j]) {
            ++i; ++j;           //继续比较后继字符
        }
        else
            j=next[j];          //模式串向右滑动
    }
    if(j>T.length)
        return i-T.length;     //匹配成功
    else
        return 0;
}

```

尽管简单模式匹配的最坏时间复杂度为 $O(mn)$ ，而 KMP 算法可达到 $O(m+n)$ ，但在一般情况下，简单模式匹配的平均执行时间往往接近 $O(m+n)$ ，因此至今仍被采用。KMP 算法仅在主串与模式串存在大量“部分匹配”时才显著优于暴力匹配，其核心优点在于主串指针不回溯。

4.2.3 KMP 算法的进一步优化

考点追踪 ▶ nextval 数组的计算 (2024)

前面定义的 next 数组在某些情况下仍存在缺陷，还可进一步优化。图 4.5 展示了一个典型示例：模式串 'aaaab' 与主串 'aaabaaaab' 进行匹配。

主串	a	a	a	b	a	a	a	a	b
模式串	a	a	a	a	b				
j	1	2	3	4	5				
next[j]	0	1	2	3	4				
nextval[j]	0	0	0	0	4				

图 4.5 KMP 算法的进一步优化示例

当 $i=4$ 、 $j=4$ 时，主串字符 s_4 与模式串字符 p_4 ($b \neq a$) 失配。若使用之前的 next 数组，则还需依次进行 s_4 与 p_3 、 s_4 与 p_2 、 s_4 与 p_1 的三次比较。然而，由于 $\text{next}[4]=3$ 且 $p_3=p_4=a$ ，同理 $\text{next}[3]=2$ 且 $p_2=p_3=a$ ， $\text{next}[2]=1$ 且 $p_1=p_2=a$ ，由于这些回退位置上的字符均与 p_4 相同，继续比较只会重复同样的失配过程，造成不必要的开销。

问题根源在于：不应出现 $p_j=p_{\text{next}[j]}$ 。理由是：当 $p_j \neq s_i$ 时，下一轮将用 $p_{\text{next}[j]}$ 与 s_i 比较；若 $p_{\text{next}[j]}=p_j$ ，则相当于用与 p_j 相同的字符去匹配已知不等的 s_i ，结果必定仍是失配。

那么，若出现 $p_j=p_{\text{next}[j]}$ ，应如何处理？

此时应不断将 $\text{next}[j]$ 替换为 $\text{next}[\text{next}[j]]$ ，直到找到某个位置 k ，使得 $p_j \neq p_k$ 或 $k=0$ 为止，修正后的新数组称为 **nextval 数组**。计算 nextval 数组的算法如下（匹配算法不变）。

```

void get_nextval(SString T, int nextval[]) {
    int i=1, j=0;
    nextval[1]=0;
    while(i<T.length) {
        if(j==0 || T.ch[i]==T.ch[j]) {
            ++i; ++j;
            if(T.ch[i]!=T.ch[j]) nextval[i]=j;
            else nextval[i]=nextval[j];
        }
        else
            j=nextval[j];
    }
}

```

KMP 算法对于初学者来说可能不太容易掌握, 强烈建议读者结合配套课程学习。

4.2.4 本节试题精选

一、单项选择题

01. 设有两个串 S_1 和 S_2 , 求 S_2 在 S_1 中首次出现的位置的运算称为 ()。
A. 求子串 B. 判断是否相等 C. 模式匹配 D. 连接
02. KMP 算法的特点是在模式匹配时, 指示主串的指针 ()。
A. 不会变大 B. 不会变小 C. 都有可能 D. 无法判断
03. 设主串的长度为 n , 子串的长度为 m , 则简单的模式匹配算法的时间复杂度为 (), KMP 算法的时间复杂度为 ()。
A. $O(m)$ B. $O(n)$ C. $O(mn)$ D. $O(m+n)$
04. 在 KMP 算法中, 用 next 数组存放模式串的部分匹配信息, 当模式串位 j 与主串位 i 比较时, 两个字符不相等, 则 j 的位移方式是 ()。
A. $j=0$ B. $j=j+1$ C. j 不变 D. $j=\text{next}[j]$
05. 在 KMP 算法中, 用 next 数组存放模式串的部分匹配信息, 当模式串位 j 与主串位 i 比较时, 两个字符不相等, 则 i 的位移方式是 ()。
A. $i=\text{next}[i]$ B. i 不变 C. $i=0$ D. $i=i+1$
06. 串 'ababaaababaa' 的 next 数组为 ()。
A. 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 9 B. 0, 1, 2, 1, 2, 1, 1, 1, 1, 2, 1, 2
C. 0, 1, 1, 2, 3, 4, 2, 2, 3, 4, 5, 6 D. 0, 1, 2, 3, 0, 1, 2, 3, 2, 2, 3, 4, 5
07. 设主串 $S='aabaaaba'$, 模式串 $T='aaab'$, 采用 KMP 算法进行模式匹配, 到匹配成功时为止, 在匹配过程中进行的单个字符间的比较次数是 ()。
A. 10 B. 9 C. 8 D. 7
08. 设主串 $S='aabaaaba'$, 模式串 $T='aaab'$, 采用改进后的 KMP 算法进行模式匹配, 到匹配成功时为止, 在匹配过程中进行的单个字符间的比较次数是 ()。
A. 9 B. 8 C. 7 D. 6
09. KMP 算法使用 nextval 数组进行模式匹配, 模式串为 $S='ababaa\underline{a}a'$, 当主串中的某个字符与 S 中的第 6 个字符失配时, S 向右滑动的距离是 ()。
A. 1 B. 2 C. 3 D. 4
10. 【2015 统考真题】已知字符串 s 为 'abaabaabacacaabaabcc', 模式串 t 为 'abaabc'。采用 KMP 算法进行匹配, 第一次出现“失配” ($s[i] \neq t[j]$) 时, $i=j=5$, 则下次开始匹配时, i 和 j 的值分别是 ()。
A. $i=1, j=0$ B. $i=5, j=0$ C. $i=5, j=2$ D. $i=6, j=2$
11. 【2019 统考真题】设主串 $T='abaabaabcabaabc'$, 模式串 $S='abaabc'$, 采用 KMP 算法进行模式匹配, 到匹配成功时为止, 在匹配过程中进行的单个字符间的比较次数是 ()。
A. 9 B. 10 C. 12 D. 15
12. 【2024 统考真题】KMP 算法使用修正后的 next 数组进行模式匹配, 模式串为 $S='aabaab'$, 当主串的某个字符与 S 的某个字符失配时, S 向右滑动的最长距离是 ()。
A. 5 B. 4 C. 3 D. 2

二、综合应用题

01. 在字符串模式匹配的 KMP 算法中, 求模式的 next 数组值的定义如下:

$$\text{next}[j] = \begin{cases} 0, & j=1 \\ \max\{k | 1 < k < j \text{ 且 } 'p_1 \cdots p_{k-1}' = 'p_{j-k+1} \cdots p_{j-1}'\}, & \text{集合不为空} \\ 1, & \text{其他情况} \end{cases}$$

- 1) 当 $j=1$ 时, 为什么要取 $\text{next}[1]=0$?
- 2) 为什么要取 $\max\{k\}$, k 最大是多少?
- 3) 其他情况是什么情况, 为什么取 $\text{next}[j]=1$?

02. 设有字符串 $S = \text{'aabaabaabaac'}$, $P = \text{'aabaac'}$ 。

- 1) 求出 P 的 next 数组。
- 2) 若 S 作为主串, P 作为模式串, 试给出 KMP 算法的匹配过程。

4.2.5 答案与解析

一、单项选择题

01. C

求子串操作是从串 S 中截取第 i 个字符起长度为 l 的子串, 选项 A 错误。选项 B、D 明显错误。

02. B

在 KMP 算法的比较过程中, 主串不会回溯, 所以主串的指针不会变小。

03. C、D

尽管实际应用中, 一般情况下简单的模式匹配算法的时间复杂度近似为 $O(m+n)$, 但它的理论时间复杂度还是 $O(mn)$ 。KMP 算法的时间复杂度为 $O(m+n)$ 。

04. D

在 KMP 算法中, 当主串的第 i 个字符和模式串的第 j 个字符不匹配时, 主串的位指针 i 不变, 将主串的第 i 个字符与模式串的第 $\text{next}[j]$ 个字符比较, 即 $j = \text{next}[j]$ 。

05. B

在 KMP 算法中, 当主串的第 i 个字符和模式串的第 j 个字符失配时, 主串指针 i 不回溯。

06. C

本题采用先求串 $S = \text{'ababaaababaa'}$ 的部分匹配值, 再求 next 数组的方法。

- 'a' 的前后缀都为空, 最长相等前后缀长度为 0。
- 'ab' 的前缀 $\{a\} \cap$ 后缀 $\{b\} = \emptyset$, 最长相等前后缀长度为 0。
- 'aba' 的前缀 $\{a, ab\} \cap$ 后缀 $\{a, ba\} = \{a\}$, 最长相等前后缀长度为 1。
- 'abab' 的前缀 $\{a, ab, aba\} \cap$ 后缀 $\{b, ab, bab\} = \{ab\}$, 最长相等前后缀长度为 2。
-

依次求出的部分匹配值见下表第 3 行, 将其整体右移一位, 低位用 -1 填充, 见下表第 4 行。

编号	1	2	3	4	5	6	7	8	9	10	11	12
S	a	b	a	b	a	a	a	b	a	b	a	a
PM	0	0	1	2	3	1	1	2	3	4	5	6
next	-1	0	0	1	2	3	1	1	2	3	4	5

选项中 $\text{next}[1]$ 等于 0, 所以将 next 数组整体加 1。

07. B

假设位序从 1 开始, 手工计算出 T 的 next 数组如下。

编号	1	2	3	4
S	a	a	a	b
next	0	1	2	3

采用 KMP 算法时：第一趟，经过 3 次比较后，发现 $S[3] \neq T[3]$ ；第二趟，用 $S[3]$ 和 $T[2]$ ($next[3]=2$) 比较，不相等；第三趟，用 $S[3]$ 和 $T[1]$ ($next[2]=1$) 比较，不相等；第四趟， $next[1]=0$ ，因此开始用 $S[4]$ 和 $T[1]$ 比较，经过 4 次比较后，匹配成功。整个匹配过程如下。

	a	a	b	a	a	a	b	a
第一趟	a	a	a	b				
第二趟		a	a	a	b			
第三趟			a	a	a	b		
第四趟				a	a	a	b	

总比较次数为 $3+1+1+4=9$ 。

08. C

假设位序从 1 开始，计算出 T 的 $nextval$ 数组如下。

编号	1	2	3	4
S	a	a	a	b
nextval	0	0	0	3

采用改进的 KMP 算法时：第一趟，经过 3 次比较后，发现 $S[3] \neq T[3]$ ；第二趟， $nextval[3]=0$ ，因此开始用 $S[4]$ 和 $T[1]$ 比较，经过 4 次比较后，匹配成功。整个匹配过程如下。

	a	a	b	a	a	a	b	a
第一趟	a	a	a	b				
第二趟				a	a	a	b	

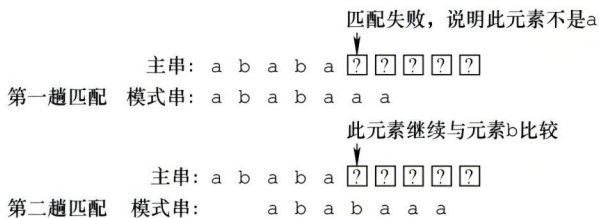
总比较次数为 $3+4=7$ 。

09. B

假设位序从 0 开始，计算出 $nextval$ 数组。当比较到 $S[j]$ 失配时，模式串向右滑动的距离为 $j - nextval[j]$ ($0 \leq j \leq 6$)，由下图可知，当 $j=5$ 时向右滑动的距离为 $5-3=2$ 。

编号 j	0	1	2	3	4	5	6
$S[j]$	a	b	a	b	a	a	a
$next[j]$	-1	0	0	1	2	3	1
$nextval[j]$	-1	0	-1	0	-1	3	1
$j - nextval[j]$	1	1	3	3	5	2	5

此外，若对 KMP 算法很熟悉，则此类题也可直接动手模拟，而不要求 $nextval$ 数组，与第 6 个字符匹配失败，说明前面的 ababa 匹配成功，S 可向右滑动 2 位继续尝试匹配。



10. C

由题中“失配 $s[i] \neq t[j]$ 时， $i=j=5$ ”，可知题中的主串和模式串的位序都是从 0 开始的（要注意灵活应变）。按照 $next$ 数组生成算法，对于 t 有

编号	0	1	2	3	4	5
t	a	b	a	a	b	c
next	-1	0	0	1	1	2